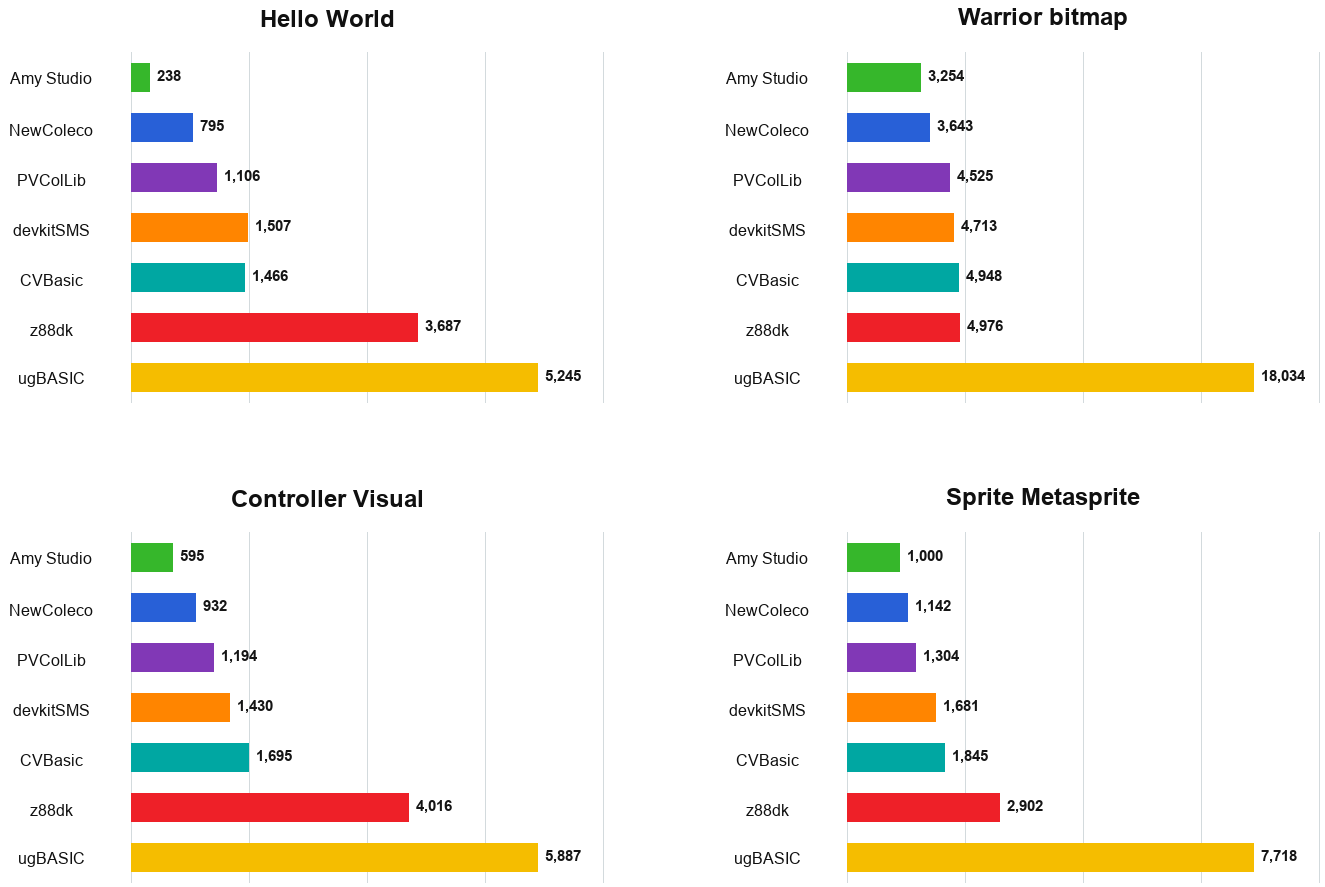
Amy Studio ColecoVision Toolchain Comparison

Measured results, capabilities, compression evidence, and development priorities

Real occupied size, excluding cartridge padding

sizes in bytes (lower is better); columns follow the four-sample total ranking



Occupied ROM size excludes cartridge padding. Lower is smaller, not automatically faster or better.

Amy Studio source and GearColeco results

The compact listings are complete. The metasprite listing omits only its six pixel-pattern blocks, which remain in the evidence archive. Screenshots come from GearColeco 1.6.8 running each measured ROM for 180 NTSC frames.

Hello World

```
text screen
backdrop sky blue
print centered at 11, "HELLO, WORLD!"
screen on
```

GearColeco screenshot

GearColeco output after 180 NTSC frames

Warrior Graphics II Bitmap

```
picture WarriorPicture:
  pattern from "@project/warrior.pattern.zx0" codec zx0
  color from "@project/warrior.color.zx0" codec zx0
end picture

show picture WarriorPicture
```

GearColeco screenshot

Exact GearColeco output; all seven tools reproduce the same target image

Visual Controller Input

```
text screen
backdrop dark blue
print centered at 8, "CONTROLLER TEST"
print centered at 11, "MOVE OR PRESS A BUTTON"
print centered at 14, "KEYPAD IS PURPLE"
screen on

InputLoop:
  wait
  if keypad(1) <> 255 then
    backdrop magenta
  elseif joypad(1).fire then
    backdrop light red
  elseif joypad(1).up then
    backdrop light green
  elseif joypad(1).down then
    backdrop dark green
  elseif joypad(1).left then
    backdrop light yellow
  elseif joypad(1).right then
    backdrop cyan
  else
    backdrop dark blue
  end if
  goto InputLoop
```

GearColeco screenshot

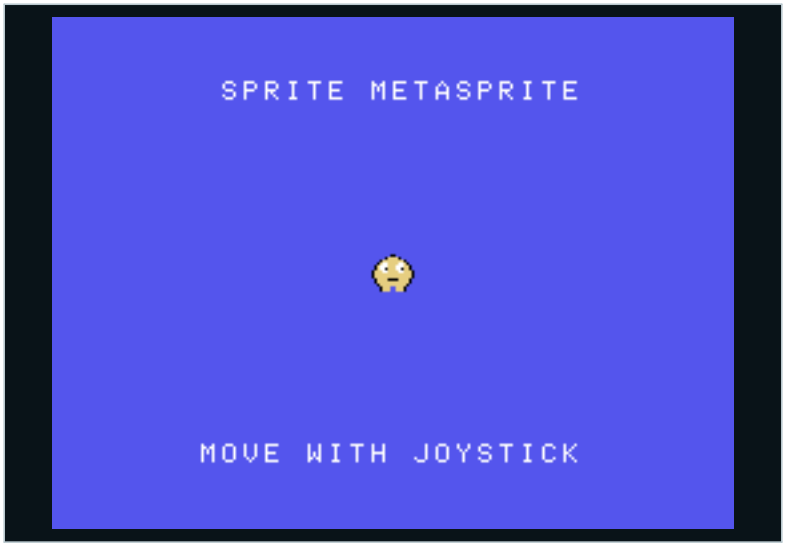
GearColeco output at the neutral controller state

Animated Three-Color Metasprite

```
u8 PlayerX = 120
u8 PlayerY = 88
u8 AnimationTick = 0
u8 AnimationFrame = 0

' Six 16x16 pattern blocks are included in the evidence archive.
sprites simple
set sprite count 3

MainLoop:
  wait
  if joypad(1).left and PlayerX > 1 then PlayerX -= 1
  if joypad(1).right and PlayerX < 239 then PlayerX += 1
  if joypad(1).up and PlayerY > 1 then PlayerY -= 1
  if joypad(1).down and PlayerY < 175 then PlayerY += 1
  AnimationTick += 1
  if AnimationTick = 8 then
    AnimationTick = 0
    AnimationFrame = 1 - AnimationFrame
  end if
  set sprite 0 to PlayerY,PlayerX,AnimationFrame * 12,15
  set sprite 1 to PlayerY,PlayerX,AnimationFrame * 12 + 4,11
  set sprite 2 to PlayerY,PlayerX,AnimationFrame * 12 + 8,1
  update sprites
  goto MainLoop
```



GearColeco screenshot

A metasprite is one visual actor assembled here from three overlapping hardware sprites

Date: 2026-09-01

Method

This comparison separates four kinds of evidence:

- **Runtime verified**: compiled for ColecoVision and checked in an emulator or self-test.
- **Build verified**: the ColecoVision source, header, library, or generated assembly was inspected.
- **Documented**: claimed by the tool's official documentation, but not yet measured here.
- **Unverified**: available somewhere in the wider tool, but not proven for ColecoVision.

Amy Studio is both a language and an integrated development environment. The other entries are primarily compilers or C development kits. IDE-only capabilities are therefore listed separately instead of being treated as language features.

The seven solutions

Solution	Primary approach	ColecoVision position
Amy Studio	ColecoVision-first Amy language and browser IDE	Original 1 KB RAM, BIOS-aware, no extra hardware by design
CVBasic	Portable integer BASIC cross-compiler	Mature native target; optional MegaCart and SGM support
z88dk `+coleco`	General Z80 C toolchain	C compiler, assembler, linker, libraries, and app packaging
ugBASIC	Portable retro BASIC compiler	Declared target with common high-level multimedia vocabulary
devkitSMS + `SGlib_CV`	SDCC C plus compact game libraries	ColecoVision adaptation of the SG/SMS development workflow
PVColLib	ColecoVision-focused SDCC C library and devkit	Native VDP, controller, sprite, sound, music, and compression APIs
NewColeco	Historical SDCC C, `CRTCV`, `CVLIB`, and GETPUT 1.1	Amy's pre-Studio ColecoVision workflow and direct ancestor of current techniques

Reproducible four-sample ROM suite

The current suite builds four runnable programs with all seven solutions:

1. a visible Hello World;
2. the Warrior Graphics II bitmap picture;
3. a visual controller monitor;
4. an animated, controller-driven three-color metasprite.

A **metasprite** is one visual actor assembled from multiple hardware sprites. The benchmark overlaps three 16x16 TMS9918 sprites for white, yellow, and black layers, remaining below the four-sprites-per-scanline limit.

Run `tools/build-five-tool-bluemsx-suite.ps1`, `tools/build-pvcollib-benchmarks.ps1`, then `node tools/build-legacy-devkit-benchmarks.mjs` for the first three samples. Their 21 ROM files are written under `build/competition/bluemsx-sample-suite`, grouped by tool. The same directory receives `occupied-sizes.csv` from `tools/report-five-tool-sample-sizes.ps1`.

Run `tools/build-sprite-metasprite-benchmarks.ps1` for the seven sprite ROMs and their stricter GearColeco VRAM/SAT oracle.

Maximum native optimization used

Solution	Setting used	Observation
Amy Studio	Experimental	Saved 1 byte on Bitmap and 2 on Controller versus Balanced; Hello was unchanged
CVBasic 0.9.2	Normal compiler; TMSColor `-z -p2` for Bitmap	No stronger compiler optimization switch was exposed; Pletter is used for its bitmap
z88dk	`+coleco -O3`; ZX0 for Bitmap	Coleco-safe direct-to-VRAM port of z88dk's ZX0 core; ZX7, MDKRLE, and raw baselines remain measured
ugBASIC 1.18	Default maximum of 16 peephole passes	Explicit 32- and 64-pass builds produced the same Hello binary
devkitSMS / SDCC 4.5	`--opt-code-size --max-allocs-per-node 100000` on the program and SGlib	Saved 60 bytes on Bitmap; Hello and Controller were unchanged
PVColLib 1.6.0 / bundled SDCC	`--opt-code-size --max-allocs-per-node 20000`	Official build flags; linked only referenced library modules
NewColeco / SDCC 3.8	`--std-c99`; original prebuilt libraries plus historical DAN2	The exact DAN2 bitmap is 626 bytes smaller than its GETPUT MDKRLE baseline

These are the strongest **native settings that were built and validated here**. Generated code was not passed through Amy's optimizer or MDL, because doing so would compare an additional external optimizer rather than each solution's normal output. Amy Experimental is appropriate for this maximum-size experiment; this table does not redefine Balanced as Amy's recommended default.

Real occupied size, excluding cartridge padding

Sample	Amy Studio	NewColeco	PVColLib	devkitSMS	CVBasic	z88dk	ugBASIC
Hello World	**238**	795	1,106	1,507	1,466	3,687	5,245
Warrior bitmap	**3,254**	3,643	4,525	4,713	4,948	4,976	18,034
Controller Visual	**595**	932	1,194	1,430	1,695	4,016	5,887
Sprite Metasprite	**1,000**	1,142	1,304	1,681	1,845	2,902	7,718
Four-sample total	**5,087**	**6,512**	**8,129**	**9,331**	**9,954**	**15,581**	**36,884**

The improved bitmap rows retain measured baselines: z88dk's raw-table ROM was 14,293 bytes, MDKRLE reduced it to 5,911 bytes, ZX7 to 5,115 bytes, and the validated ZX0 path to 4,976 bytes; the legacy devkit's GETPUT MDKRLE ROM was 4,269 bytes before DAN2 reduced it to 3,643. Both replacements reproduce the original pattern table, color table, and all 49,152 pixels exactly. The installed ugBASIC 1.18 Coleco target accepts `LOAD IMAGE ... COMPRESSED`, but Warrior produces the same 18,034-byte ROM and identical hash as `NONE`: MSC1 is discarded when it does not shrink the converted resource. The RLE image branch is compiled only for C128, not Coleco.

The exact legacy payload comparison is MDKRLE 3,687 bytes, DAN1 2,903, DAN2 2,897, and DAN3 2,891. DAN2 remains the linked benchmark: DAN3 saves only six payload bytes before decoder cost, while the historical DAN2 SDCC wrapper is already validated end to end.

`Occupied` includes the cartridge header, linked runtime, program code, and ROM data. It does not include the `$00`/`$FF` bytes added to reach an 8, 16, or 32 KB cartridge image. The report never guesses by removing repeated final bytes:

- Amy uses the assembled ROM length;
- CVBasic uses `ROM_END - $8000` from the assembler listing;
- z88dk uses its unpadded linked binary;
- ugBASIC uses its generated code and data binaries;

- devkitSMS uses the occupied Intel HEX address span above `$8000`.
- PVColLib uses the occupied Intel HEX address span above `$8000`.
- the legacy devkit uses the complete unpadded binary emitted from its Intel HEX link.

Runtime and comparability verdict

Sample	Runtime result	Verdict
Hello World	Seven ROMs complete 180 GearColeco frames	Stable startup
Warrior bitmap	Seven native pipelines render the same 256x192 image	`0 / 49,152` pixels differ
Controller Visual	Six pass injected neutral, keypad, UP, FIRE, and release states	Partial: ugBASIC does not update VDP R7
Sprite Metasprite	Seven pass the same VRAM and sprite-table checks	Exact patterns, layers, and priority

PVColLib and NewColeco use `$F0` white-on-transparent text in the controller fixture instead of their customary `$F1` white-on-black. This presentation-only normalization exposes the changing VDP R7 backdrop without changing controller logic or occupied size.

The bitmap test uses each tool's validated native workflow: Amy ZX0, CVBasic Pletter, z88dk ZX0, ugBASIC image resources, devkitSMS aPLib, PVColLib RLE, and NewColeco DAN2. CVBasic's tables differ internally, but TMS9918 can encode the same pixels several ways. All seven framebuffers are exact. PVColLib Pletter failed VRAM validation and is excluded; codec rankings use the separate payload test.

For z88dk, MDKRLE reaches VRAM at frame 93, ZX0 at 132, and ZX7 at 138. ZX0 saves 935 bytes over MDKRLE and 139 over ZX7; MDKRLE loads faster. A separate attempt to transplant the SMS aPLib decoder into the z88dk fixture failed exact VRAM validation and is excluded. The native devkitSMS aPLib fixture remains exact when frame interrupts are disabled during decompression.

The initial ugBASIC mismatch came from the wrong RGB palette. With its target palette, all four corpus pictures render exactly.

Graphics II bitmap compression ratios

Each picture is exactly 12,288 RAW bytes (6,144 Pattern + 6,144 Color). Percentages are compressed payload / RAW payload; lower is better. Decoder code is excluded because it is linked once and may serve several assets. Every measured stream round-trips exactly; DAN3 uses its full-search best-size setting. ZX0 Classic uses the official z88dk ZX0 v1.5 compressor; its Coleco VRAM decoder was separately runtime-verified in the Warrior benchmark. Classic and modern ZX0 use incompatible stream formats, but their payload sizes match for every picture here, so one ratio column represents both.

LZ-family ratios

Picture	ZX0 Classic / modern	ZX1	ZX2	ZX7	aPLib	Pletter	BitBuster	LZF
Cake	24.07%	25.33%	25.42%	25.26%	24.85%	25.29%	25.44%	29.48%
Commando	43.21%	45.58%	45.18%	44.86%	44.39%	44.82%	45.07%	49.93%
Warrior	23.19%	24.12%	25.38%	24.28%	24.19%	24.32%	24.43%	26.00%
Barbarian	34.42%	36.11%	36.39%	35.31%	35.45%	35.31%	35.51%	39.10%
421 title	8.59%	8.97%	9.08%	9.02%	8.94%	9.03%	9.16%	11.01%
421 credits	22.68%	23.56%	23.40%	24.18%	23.50%	24.22%	24.35%	26.34%
Dacman title	8.01%	8.33%	8.49%	8.38%	8.24%	8.42%	8.50%	9.62%
Dacman info	10.25%	10.47%	11.34%	11.06%	10.38%	11.08%	11.19%	12.51%
Dacman 2 title	9.56%	10.04%	10.13%	9.99%	9.99%	10.02%	10.12%	11.82%

Picture	ZX0 Classic / modern	ZX1	ZX2	ZX7	aPLib	Pletter	BitBuster	LZF
Chateau title	13.31%	13.70%	13.77%	14.07%	13.88%	14.08%	14.18%	15.53%
Arcade Trio title	24.41%	25.82%	25.53%	25.19%	25.06%	25.20%	25.33%	29.26%

DAN and RLE-family ratios

Picture	DAN1	DAN2	DAN3 Best	Nibble	MDK-RLE
Cake	24.46%	24.41%	24.45%	28.92%	33.11%
Commando	43.33%	43.23%	43.29%	51.23%	59.29%
Warrior	23.62%	23.58%	23.53%	27.22%	30.00%
Barbarian	35.34%	35.29%	34.88%	41.32%	44.78%
421 title	8.76%	8.68%	9.05%	11.95%	13.96%
421 credits	22.96%	22.94%	22.93%	25.75%	28.91%
Dacman title	7.89%	7.91%	8.41%	9.63%	11.31%
Dacman info	10.38%	10.36%	10.51%	13.31%	14.59%
Dacman 2 title	9.53%	9.51%	9.71%	13.54%	16.42%
Chateau title	13.44%	13.44%	13.49%	17.01%	18.95%
Arcade Trio title	24.58%	24.36%	24.63%	30.24%	35.71%

Amy direct-to-VRAM decompressor sizes

These are assembled routine bytes, excluding compressed data and common program startup code. A routine is linked once and can decode any number of assets using that codec.

Codec	Routine bytes
ZX0 modern	133
ZX1	127
ZX2	115
ZX7	136
aPLib Compact	244
Pletter	212
BitBuster	166
LZF	117
DAN1	205
DAN2	212
DAN3 Best	205
Nibble	115
MDK-RLE	46

Representative first-use totals

Each value is the complete compressed Pattern + Color payload plus one Amy direct-to-VRAM decompressor. It is not the full ROM size. Commando, Warrior, and Dacman title represent a difficult, middle, and highly compressible case in this corpus.

Codec	Commando	Warrior	Dacman title
ZX0 modern	5,443	2,982	1,117
ZX1	5,728	3,091	1,151
ZX2	5,667	3,234	1,158
ZX7	5,649	3,120	1,166
aPLib Compact	5,699	3,217	1,256
Pletter	5,720	3,200	1,247
BitBuster	5,704	3,168	1,211
LZF	6,253	3,312	1,299
DAN1	5,529	3,108	1,174
DAN2	5,524	3,109	1,184
DAN3 Best	5,525	3,096	1,238
Nibble	6,410	3,460	1,298
MDK-RLE	7,331	3,733	1,436

Amy Studio's aPLib Compact routine is 244 bytes under every optimization profile. It replaces the previous 348-byte unrolled routine without changing the stream format or payload, and preserves IX and IY for safe return to Amy and BIOS code. MDK-RLE remains the smallest decoder at 46 bytes, but its payloads are much larger here.

The complete payload counts and ratios are preserved in **`competition/benchmarks/compression/bitmap-codec-ratios.csv`**; decoder and representative first-use totals are in **`bitmap-codec-first-use.csv`**. The earlier four-picture framebuffer checks remain in the evidence archive: all twelve converter ROMs differed by **`0 / 49,152`** pixels.

Observations supported by this suite

- Amy has the smallest Hello and controller-monitor footprint because its Coleco BIOS-aware

runtime is selected from actual source capabilities.

- Amy wins this exact full-picture ROM measurement. CVBasic's internally different TMS9918 tables

render the same pixels, but its Pletter streams and larger runtime produce a 4,948-byte result versus Amy's 3,254 bytes.

- General C/BASIC portability runtimes have a visible fixed cost on tiny programs; this does not

prove they remain proportionally larger in complete games.

- Cartridge file length is packaging evidence, not occupied-code evidence. A 32 KB z88dk file is

not automatically a 32 KB program.

- Optimization claims require runtime validation. A smaller file without matching behavior is

rejected rather than counted as a win.

- Four samples are insufficient to declare one compiler universally smaller or faster. RAM,

worst-frame cycles, build latency, sound behavior, sprite pressure, and gameplay logic remain separate measurements.

Preliminary capability matrix

Legend: **Yes** = verified or explicit target support; **Partial** = manual or narrower support; **Pending** = must still be proven specifically on ColecoVision. * marks a benchmark adaptation written here, not a native ColecoVision direct-to-VRAM path supplied by that solution.

Capability	Amy Studio	CVBasic	z88dk	ugBASIC	devkitSMS / SGlib_CV	PVColLib
8/16-bit integers	Yes	Yes	Yes	Documented	Yes, through SDCC	Yes, through SDCC
32-bit integers	Yes, runtime tested	No native BASIC type	Yes	Documented; target cost pending	Yes, through SDCC	Yes, through SDCC
Fixed-point	Yes, 8.8 and 16.16	No	Library/manual	Documented; target proof pending	Manual C/library code	Manual C/library code
Records/structures	Yes	No	Yes	Documented custom types	Yes	Yes
2D arrays	Yes	No native 2D syntax	Yes	Documented	Yes	Yes
RAM overlays	Yes, first-class	Manual	C union/manual layout	Pending	C union/manual layout	C union/manual layout
Functions with values and returns	Yes	Limited BASIC procedures/DEF FN	Yes	Yes	Yes	Yes
Inline Z80 assembly	Yes	Yes	Yes	Documented	Yes	Yes
TMS9918 screen and tile operations	Yes	Yes	Yes	Yes, target proof incomplete	Yes	Yes
Hardware sprite API	Yes	Yes	Low-level/library	Yes, exact target surface pending	Yes	Yes, 32-entry SAT
Four-sprite scanline mitigation	Yes, stable ranges and flicker	Yes, sprite flicker	Manual	Pending	Manual/library-dependent	Yes, NMI sprite swapping
Keypad	Yes	Yes	Library/manual	Pending	Yes/target library	Yes, both ports
Spinner / Roller Controller	Yes	Yes	Not confirmed	Pending	Not confirmed	Yes, both ports
Held, pressed, and released input	Yes	Held; edges are manual	Manual	Manual edges over `JOY`	Yes, computed from NMI snapshots	Manual edges over NMI snapshots
Coleco PSG sound	BIOS tables, Tiny Sound, DSOUND	Sound/music commands	Sound libraries	Sound commands, target proof pending	PSGlib_CV	BIOS-style sound tables and sequenced music
Direct-to-VRAM compression	Thirteen active codecs; ZX0 Classic measured separately	Pletter	RAM APIs; ZX0, *ZX1*, ZX2, *and ZX7* benchmark VRAM ports	Resource conversion; RAM-oriented compression	ZX7 and aPLib	RLE, Pletter, DAN1/2/3
ROM banking	Intentionally no	Yes	Yes	Pending	SMS workflow has banking; CV support pending	MegaCart tools and examples
Source-level Coleco debugger	Integrated	External emulator	External debugger/emulator	External or IDE-dependent	External debugger/emulator	External debugger/emulator
Rewind, breakpoints, VRAM/RAM inspection	Integrated	External	External	External	External	External
Graphics editors and project assets	Integrated	Separate tools	Separate tools	Conversion-oriented IDE/tools	Separate asset tools	`gfx2col` and separate asset tools
Automated ROM behavior tests	Integrated project pipeline	Not supplied as one system	Can be assembled manually	Not established	Not established	Not established

Detailed language and runtime evidence

Data model

Area	Amy Studio	CVBasic	z88dk	ugBASIC	devkitSMS	PVColLib
Integer widths	Explicit signed/unsigned 8/16/32	8/16, signed or unsigned	C 8/16/32	Backend defines signed/unsigned 8/16/32	SDCC C 8/16/32	SDCC C 8/16/32
Fractional math	`fixed`, `ufixed`, `fixed32`, `fp5`	None	C/library float and fixed libraries	`VT_FLOAT` exists; Coleco cost pending	SDCC float/manual fixed	SDCC float/manual fixed
Decimal scores	Packed BCD 1-12 digits	Manual	Manual/library	Manual	Manual/library	Manual/library
Aggregates	Nested records, record arrays	None	struct/union	Typed arrays and internal resource types; user-structure surface pending	struct/union	struct/union
Multidimensional arrays	Primitive 2D arrays	One-dimensional only	Native C	Typed arrays; dimensional coverage pending	Native C	Native C
Mutually exclusive RAM	First-class overlays/scenes	Manual aliases	union/linker/manual	Banking/resource allocator; no equivalent proven	union/linker/manual	union/linker/manual
Dynamic strings	Deliberately fixed-buffer oriented	Mostly literals/printing	C strings and allocation libraries	First-class strings	C strings; risky in 1 KB RAM	C strings; risky in 1 KB RAM

Amy's data model is the most game-oriented for stock ColecoVision RAM. C remains the most general, but generality does not provide automatic lifetime analysis or a debugger-aware overlay. ugBASIC has a broad internal type system; its actual ColecoVision ROM/RAM cost still requires compilation.

Control, timing, and code organization

Area	Amy Studio	CVBasic	z88dk	ugBASIC	devkitSMS	PVColLib
Procedures/functions	Typed values, `ref`, returns, recursion	Procedures without value parameters; `DEF FN` expression macros	Full C	Procedures/functions documented	Full C	Full C
State machines	Typed zero-RAM dispatch	Manual `SELECT`/labels	Manual enum/switch/table	Manual control constructs	Manual enum/switch/table	Manual enum/switch/table
VBlank	`wait`, frame hooks, timers	`WAIT`, `ON FRAME`	HALT/NMI hooks/manual	`WAIT VBL`, `EVERY`, timers	`SG_waitForVBlank`, frame handler	`vdp_waitvblank`, user `nmi` hook
Parallel animation	Not yet first-class	Manual	Manual/interrupt	`ANIMATION`, `ANIMATE`, `MOVE`, paths and multitasking source present	Manual frame service	Manual NMI/frame service
Inline assembly	Amy ASM bridge	`ASM`, `CALL`, `USR`	Native inline/external ASM	Supported	SDCC inline/external ASM	SDCC inline/external ASM

ugBASIC is the strongest source of ideas for a future optional Amy animation service. It also shows the risk: animations create state variables, thread handles, paths, delays, signals, and background-preservation storage. Amy should not copy that surface until exact RAM and cycle costs are visible and the entire service disappears when unused.

TMS9918 graphics and sprites

Area	Amy Studio	CVBasic	z88dk	ugBASIC	SGlib_CV	PVColLib
Text/tile/bitmap modes	Coleco-specific commands and editors	Modes and direct VRAM commands	Generic TMS9918/graphics libraries	TMS9918 backend and resource conversion	Tile, bitmap, VDP and VRAM functions	Text, bitmap, VDP and VRAM functions
Pixel/line/circle	Yes, programmer controls NMI-safe batching	Yes	Generic graphics	TMS9918 drawing backend	Pixel/bitmap helpers	Low-level VRAM/bitmap support
Sprites	Fields, clipping helpers, movement, editor	Define/place sprites	Low-level/library	Sprite conversion and TMS9918 operations	SAT builder, clipping, finalize/copy	32-entry SAT, update and fast-update APIs

Area	Amy Studio	CVBasic	z88dk	ugBASIC	SGlib_CV	PVCollib
Collision	Hitboxes, tile groups, hardware status	VDP status/manual	Hardware/manual	Hardware collision/hit backend	VDP status/manual	`spr_collide` hitboxes and VDP status
Scanline overflow	Explicit flicker and stable priority ranges	Sprite flicker	Manual	No verified policy	Manual ordering	`spr_getentry` swaps sprites each NMI
Asset authoring	Integrated bitmap/tile/sprite/frame editors	Separate tools	Separate tools	Automatic image conversion	External asset tools	`gfx2col` and external tools

Amy's lead is the complete authoring/debugging workflow and explicit TMS9918 policy. ugBASIC's automatic modern-image conversion is substantial, while SGlib_CV offers a compact C SAT workflow.

Controllers

Capability	Amy Studio	CVBasic	z88dk	ugBASIC	SGlib_CV	PVCollib
Two directions/fire ports	Yes	Yes	`joystick(1/2)`	`JOY(0/1)` backend	NMI snapshots both ports	`joypad_1/2`, four fire bits
Two keypad ports	Yes	Yes	`joystick(3/4)` high byte	`SCANCODE` path; exact port behavior pending	`SG_readCVNumPad`	`keypad_1/2`
Pressed/released	First-class per property	Manual previous state	Manual previous state	Manual previous state	First-class status helpers	Manual previous-state comparison
Spinner/Roller	Yes	Yes	Not found in Coleco target	Not found yet	Not found	`spinner_1/2`, reset and enable APIs
Automatic minimal backend	Yes, capability selected	Runtime-integrated	Linker includes referenced routines	Deployment system includes used modules	Library/NMI input service	Linker extracts referenced modules

Sound and music

Area	Amy Studio	CVBasic	z88dk	ugBASIC	devkitSMS	PVCollib
SN76489 tones/noise	BIOS sound tables and direct formats	`SOUND`	PSG libraries/manual ports	SN76489 backend	PSGlib_CV	BIOS-style sound tables
Sequenced music	BIOS format and Tiny Sound	`MUSIC`, simple/full players	External/player libraries	`MUSIC` backend	PSG streams; looping/status	NMI-serviced music sequences
Concurrent SFX	Coleco table areas/Tiny Sound	Channel depends on music mode	Library-dependent	Pending measurement	PSG SFX channels and frame service	Four music areas plus sound areas 5+ for SFX
Digital samples	DSOUND	No first-class support	Manual/custom	Pending	No first-class support found	No first-class support found
Authoring	Amy's CV Sound Studio exists; Studio integration incomplete	Note-oriented BASIC source	External VGM/tools	Source/resource conversion	External PSG tools	External table/asset tools

Amy has the broadest playback formats, but CVBasic currently has the simplest music source syntax. The most valuable Amy improvement remains visual, reliable creation of Coleco BIOS/Tiny Sound tables rather than another playback format.

Compression evidence

Compression must be scored as **`compressed payload + linked decompressor`**, with destination and cycles. Counting a host compressor without a ColecoVision decoder is invalid.

All thirteen integrated Amy codecs were also compiled as separate Balanced ROMs and run for 180 NTSC frames in GearColeco. Every ROM reproduced Warrior's 6,144 Pattern bytes and 6,144 Color bytes exactly in VRAM. Run **`node tools/test-integrated-codec-vram-roms.mjs`** to repeat this check.

Solution	Confirmed formats	Direct VRAM status	Integrated selection
Amy Studio	ZX0, ZX1, ZX2, aPLib, ZX7, Pletter, DAN1/2/3, LZF, BitBuster, MDK-RLE, Nibble	ColecoVision paths, including workspace-based formats	Browser comparison/import and asset metadata
CVBasic	Pletter	`DEFINE CHAR/COLOR/SPRITE/VRAM PLETTER`	Explicit source keyword
z88dk	ZX0/1/2/7 and aPLib families, multiple speed/size decoders	Stock decoders target RAM; ZX0 *and ZX7* Coleco VRAM adaptations verified here	Manual headers/linking and host tools
ugBASIC	MSC1 and RLE types in compiler source	MSC1 image fallback verified; RLE is not implemented for Coleco	Resource compiler can choose compression when it wins
SGlib_CV / SMSlib routine	ZX7 and aPLib	ZX7 API; aPLib direct-to-VRAM exact with frame IRQ disabled	Manual host compression and C asset inclusion
PVColLib	RLE, Pletter, DAN1/2/3	RLE direct-to-VRAM verified exactly here; other paths remain separate candidates	`gfx2col` conversion and C asset inclusion
NewColeco	GETPUT 1.1 MDK-RLE	Direct-to-VRAM verified exactly on Warrior	External CVPaint/asset tools and C data inclusion

z88dk's public ZX0 integration request dates from February 2021, consistent with ZX0/ZX7 being available there by spring 2021. Its bundled compressor identifies itself as ZX0 v1.5 and emits the official classic v1 format. Amy Studio's current `zx0` codec emits the later official v2 format by default. The streams are intentionally not interchangeable. z88dk's stock routines target RAM; this benchmark adds only ColecoVision TMS9918 direct-to-VRAM adaptations and validates their output independently.

Codec naming and integration policy

Candidate	Honest Amy name	Current evidence	Integration condition
ZX0 modern	`ZX0` / codec `zx0`	Existing v2 browser encoder and Coleco VRAM decoder	Keep as the default
ZX0 classic	`ZX0 Classic (v1)` / proposed codec `zx0v1`	z88dk v1.5 compressor plus exact benchmark VRAM port	Explicit extension and cross-format rejection tests
ZX1	`ZX1` / codec `zx1`	Byte-identical browser encoder; exact four-picture GearColeco VRAM proof	Integrated; measure Coleco cycle cost
ZX2	`ZX2` / codec `zx2`	Byte-identical browser encoder; exact eleven-picture round-trip and five-profile GearColeco VRAM proof	Integrated; measure Coleco cycle cost
aPLib	`aPLib` / codec `aplib`	Bidirectional appack parity and exact Amy/GearColeco VRAM ROM	Integrated; keep NMI-safe upload and attribution explicit
MSC1	`MSC1`	ugBASIC discards it for Warrior when it gives no gain	Useful Coleco corpus wins and a VRAM strategy

A host compressor alone is insufficient. An Amy codec requires round-trip tests, exact GearColeco VRAM output, decoder cost, explicit destination semantics, malformed-stream failure tests, documentation, and attribution. RAM-only APIs and locally written VRAM ports must remain labelled.

Current verdict: Amy leads in codec breadth and ColecoVision workflow; devkitSMS has verified ZX7 and aPLib paths, PVColLib has a verified RLE path, and CVBasic has a concise Pletter path. On Warrior and Cake, official ZX1, ZX2, and aPLib do not beat Amy's ZX0 first-use ROM size. aPLib nevertheless cuts the devkitSMS Warrior ROM from 13,680 raw bytes to 4,713 occupied bytes. ZX1 is now an integrated Amy codec with a 127-byte direct-to-VRAM decoder and exact Cake, Commando, Warrior, and Barbarian runtime proofs. Its remaining question is measured cycle cost. ugBASIC's MSC1 is a valid comparison candidate, not ten separate codecs, but it does not improve the Warrior resource.

What MSC1 actually is

MSC1 is ugBASIC's compact block-repetition format, not another general ZX0-style LZ codec. A stream contains literal blocks of 1-127 bytes and back-references that repeat one four-byte sequence up to 32 times. The back-reference offset is limited to roughly 1 KB, and a zero token ends the stream. This favors maps, records, planar graphics, and other data with repeated groups of four bytes.

The current Z80 decoder is simple and Apache-2.0 licensed, but writes to ordinary memory with `LD (DE),A`. On a stock 1 KB ColecoVision it cannot directly expand a 6 KB pattern or color table. More importantly, ugBASIC's generic `LOAD("file") COMPRESSED` implementation only invokes MSC1 when the target declares expansion banks. The standard Coleco target does not, so the keyword is accepted but ignored there. Five raw/compressed resource pairs produced byte-identical generated code and data. This is therefore not presently a general Coleco capability comparable to Amy's direct-to-VRAM codecs.

MSC1 can still be selected by some ugBASIC image/resource conversion paths. A fair evaluation of that narrower feature needs a direct-to-VRAM proof and a mixed corpus: bitmap tables, name tables, tile sets, sprite frames, level maps, record arrays, and sound data. Only candidates that reduce `payload + linked decoder`, round-trip exactly, and pass GearColeco should be added.

aPLib integration status

aPLib has three useful pieces already available locally: the official host compressor, Amy's JavaScript beam-search encoder, and direct-to-VRAM Z80 implementations in z88dk/devkitSMS. The devkitSMS routine uses the Coleco-compatible `$BF/$BE` VDP ports, but is explicitly unsafe while the display is active and lacks the Coleco BIOS protection used by `SG_decompressZX7toVRAM`.

The JavaScript encoder now passes bidirectional differential checks against official aPLib on real TMS9918 tables and structured, incompressible, repetitive, short, and boundary-sized data. Its exact-cost planner produces 3,054 bytes for Cake and 2,973 for Warrior, versus official raw payloads of 3,088 and 2,975. Official `appack` still wins some individual resources, so neither encoder universally dominates.

Amy now ships the browser encoder and a 244-byte compact public-domain SMSlib-derived ColecoVision VRAM decoder. `decompress aplib Source to vram.pattern` accepts a raw `.aplib` stream and the existing VRAM upload wrapper provides the same NMI-safe contract as other Amy LZ codecs. A compiled Amy ROM reconstructed Warrior's 6,144-byte pattern and color tables exactly in GearColeco under all five optimization profiles. The Balanced test ROM occupied 3,517 bytes with 2,973 compressed data bytes. The official aPLib executable is used only for differential testing and is not redistributed.

The first reproducible size results and the rules for the cross-tool runtime comparison are in `competition/benchmarks/compression/README.md`. On two real 12,288-byte TMS9918 pictures, ZX0 has the smallest Amy first-use ROM total after its current 133-byte decompressor estimate is included. This is a size result, not a speed verdict; Z80 cycle measurements remain required.

Memory, ROM size, and banking

Area	Amy Studio	CVBasic	z88dk	ugBASIC	devkitSMS	PVColLib
Stock 1 KB RAM focus	Core design, RAM estimates, overlays	Yes, global/static model	Configurable CRT/C runtime	Backend manages runtime/resources	SDCC/static library model	SDCC/static library model
Dead helper elimination	Capability-driven generation	Compiler-generated runtime	Linker sections/libraries	Deploy-on-use modules and target optimizer	Linker library extraction	Linker library extraction
Optimizer	Five profiles plus runtime corpus	Z80 optimizer and peepholes	sccz80/zsdcc optimizers	Coleco-specific optimizer source	SDCC optimizer/peepholes	SDCC size optimization
Beyond 32 KB	Deliberately excluded	MegaCart up to 1 MB	Coleco banking/toolchain	Target support not yet proven	MegaCart and banked functions documented	MegaCart tools and example verified
Debug-aware RAM names	Yes, including overlay aliases	Assembly labels	Map/debug symbols	Generated symbols	Map symbols	Map symbols

Banking is a capability advantage for CVBasic, z88dk, and devkitSMS, but not a feature Amy intends to adopt: Amy explicitly preserves the original unexpanded-hardware philosophy. The comparison should describe this limitation honestly without turning it into a roadmap requirement.

Stock baseline versus expanded hardware

The size and runtime ranking above targets an original ColecoVision: TMS9918A video, SN76489 sound, 1 KB system RAM, Coleco BIOS, and an unbanked cartridge whenever possible. Optional hardware is credited separately because it changes the machine available to the programmer:

Extension	Verified or documented advantage	Scope in this comparison
MegaCart / bank switching	CVBasic, z88dk, devkitSMS, and PVColLib can exceed the normal cartridge space	Valid capability; excluded from stock-ROM size ranking
F18A	PVColLib provides dedicated APIs and examples for enhanced video hardware	Valid for modified or compatible clone systems; out of scope for ColecoVision-exclusive titles
SGM / AY-3-8910-compatible sound	CVBasic and PVColLib provide explicit SGM paths; PVColLib also detects SGM RAM and ADAM	Valid expansion/clone capability; out of scope for the stock sound ranking
Extra RAM	Available through SGM, ADAM, and compatible clones depending on the tool/runtime	Report separately; never count it as stock 1 KB RAM

This separation lets every solution show its extended-hardware strengths without weakening Amy Studio's deliberate original-hardware target.

Development experience

Area	Amy Studio	CVBasic	z88dk	ugBASIC	devkitSMS	PVColLib
Browser IDE	Integrated	No official integrated IDE	No	IDE/web options advertised	No	No
Source breakpoints	Yes	External emulator	External debugger	IDE-dependent/unverified	External emulator	External emulator
ASM stepping/rewind	Integrated GearColeco workflow	External	External	External	External	External
RAM/VRAM/symbol inspection	Integrated	External	External	External	External	External/map symbols
Cycle profiling	Integrated	External/manual	External/manual	Unverified	External/manual	External/manual
Project graphics editors	Integrated/configurable	Separate tools	Separate tools	Resource conversion/IDE	Separate tools	`gfx2col`/separate tools
Automated ROM tests	Corpus, checkpoints, runtime harness	Not integrated	Buildable manually	Not established	Not established	Not established

This is Amy Studio's decisive advantage. A fair comparison should still label command-line-only Amy scripts separately from features directly accessible in the IDE.

Amy Studio leads in ColecoVision integration

Amy's strongest distinction is not one isolated keyword. It connects source editing, asset conversion, compression, assembly optimization, ROM execution, source breakpoints, rewind, memory and VRAM inspection, controller configuration, profiling, and automated tests in one ColecoVision-focused workflow. Its overlays, typed state machines, BCD, fixed-point support, runtime-checked wide integers, collision helpers, and BIOS-aware input selection are also substantial language-level strengths.

CVBasic leads in portability and established BASIC simplicity

CVBasic has a compact QBasic-like surface, a mature ColecoVision backend, many complete game examples, spinner support, sprite flicker support, music commands, and optional ROM banking. It also targets many related machines. It does not offer Amy's records, overlays, wide numeric model, integrated debugger, or asset/debug pipeline.

Official source: [nanochess/CVBasic](https://github.com/nanochess/CVBasic)

z88dk leads in general C and toolchain breadth

z88dk provides full C data structures, pointers, mature compilers, assemblers, linkers, libraries, compression utilities, and many Z80 targets. This flexibility also exposes more low-level choices to the programmer. Its generic graphics or library facilities must not be mistaken for an Amy-style ColecoVision game API without target-specific proof.

Official source: [z88dk/z88dk](https://github.com/z88dk/z88dk)

ugBASIC has the most ambitious portable high-level vocabulary

ugBASIC documents nonblocking animation, movement, paths, image conversion, sprites, sound, and many modern BASIC constructs. This makes it especially relevant when considering future Amy animation ergonomics. However, its multi-target manual describes capabilities that can differ by backend. Generated ColecoVision Z80 and runtime behavior must be measured before declaring those features equivalent.

Official sources: [ugBASIC site](https://ugbasic.iwashere.eu/) and [spotlessmind1975/ugbasic](https://github.com/spotlessmind1975/ugbasic)

devkitSMS offers a compact C game-library workflow

devkitSMS combines SDCC with small game-oriented libraries and asset tools. **`SGlib_CV`** and **`PSGlib_CV`** make it relevant to ColecoVision even though much of the surrounding documentation is SMS-oriented. Only APIs present in the CV headers and libraries should enter the final score.

The local **`SGlib_CV`** source confirms that its ColecoVision NMI scans direction/fire mode and numpad/right-trigger mode, preserves current and previous states, and implements **`SG_getKeysStatus`**, **`SG_getKeysPressed`**, **`SG_getKeysHeld`**, **`SG_getKeysReleased`**, and **`SG_readCVNumPad`**. These input capabilities are source-verified for ColecoVision, not inferred from SMSlib.

Official source: [sverx/devkitSMS](https://github.com/sverx/devkitSMS)

PVColLib offers broad ColecoVision-focused C APIs

PVColLib is a ColecoVision-focused SDCC library and development kit. Its official repository includes the compiler toolchain, VDP and controller APIs, sprites, sound/music support, compression routines, MegaCart, SGM, Phoenix, and F18A facilities. The three stock fixtures here use its official build flags and linked library. Local headers and examples verify both controller ports and keypads, both spinners, 32 sprites, NMI sprite swapping, hitbox collisions, BIOS-style sound tables, sequenced music with concurrent SFX areas, direct VRAM operations, MegaCart, SGM, Phoenix, and F18A support. Its RLE bitmap path was validated byte-for-byte in VRAM; the available Pletter and DAN paths are not credited with a benchmark size until a compatible stream passes the same exact runtime test.

Official source: [alekmaul/pvcollib](https://github.com/alekmaul/pvcollib)

NewColeco remains a strong historical baseline

The recovered SDCC branch uses Amy's original **`CRTCV`**, **`CVLIB`**, and GETPUT 1.1 libraries. Its prebuilt ASxxxx objects link successfully with the locally available SDCC 3.8 toolchain after using the modern **`.rel`** extension; no ABI or library source change was required. The three new fixtures complete 180 NTSC frames in GearColeco. Warrior is decompressed directly to VRAM with GETPUT's MDK-RLE routine and matches all 12,288 target table bytes and all 49,152 pixels.

This is not an unrelated seventh competitor: it is the documented historical ancestor of Amy Studio. Its 795-byte Hello, 3,643-byte DAN2 bitmap, and 932-byte controller monitor show that the old BIOS-aware and link-only-what-is-used philosophy was already effective. Amy Studio improves those results while adding the Amy language, integrated assets, diagnostics, and debugging.

Next measurement suite

The comparison becomes stronger as equivalent game behaviors are implemented seven times. Current status and order:

1. **Controller snapshot: built, semantic injection pending.** Inject directions, both fire

buttons, keypad, press, hold, and release; assert the result bytes instead of only booting.

2. **Sprite/metasprite stress: next.** Move eight visible sprites and two multi-sprite actors

across one scanline; record SAT ordering, flicker policy, ROM, RAM, and worst-frame cycles.

3. **Tile animation.** Update a small Graphics II region every frame without corruption and

measure VRAM bytes per frame.

4. **State update.** Run an actor array, collision checks, timers, and state dispatch.
5. **Sound.** Start, loop, stop, and switch one ColecoVision PSG sequence.
6. **Compression: payload sizes measured; cycle suite pending.** Compare identical

pattern/color/name data including linked decoder bytes, destination, and cycles.

7. **Visible Hello: complete.** Keep it separate from the minimal runtime fixture so font/text

costs remain explicit.

The first fixture is now in `competition/benchmarks/controller-input`. Initial build facts:

- Amy: Off 413, Safe 409, Balanced 408, Aggressive 408, and Experimental 406 ROM bytes. The

fixture has five explicit result bytes plus Amy's selected runtime state. All controller capabilities are selected from actual source usage. Runtime input injection is still required before these five builds receive the stronger runtime-verified status.

- z88dk `+coleco -O2`: 2,117 bytes of useful binary before its cartridge image is padded to

32,768 bytes. Its CRT/BSS map reaches `$717A`; the five result bytes occupy `$701F-$7023`.

- CVBasic, ugBASIC, and devkitSMS are now locally built and measured below.

Controller fixture ROM results

The build keeps all five ROMs under `build/competition/controller-input/` so they can be loaded manually in Amy Studio ROM TEST & DEBUG or another ColecoVision emulator.

Tool	Occupied ROM	ROM file	Entry	GearColeco
Amy Studio Balanced	**408**	408	`$8120`	120-frame boot PASS
devkitSMS / SDCC 4.5	1,337	16,384	`$8024`	120-frame boot PASS
CVBasic 0.9.2	1,483	8,192	`$853D`	120-frame boot PASS
z88dk `+coleco -O2`	2,117	32,768	`$802A`	120-frame boot PASS
ugBASIC 1.18	5,337	16,384	`$82BF`	120-frame boot PASS

`Occupied ROM` compares compiler and runtime output. `ROM file` is the downloadable file size and includes each toolchain's padding. CVBasic is measured from `ROM_END`, z88dk from its unpadded linked binary, ugBASIC from its code and data binaries, and devkitSMS from its Intel HEX span. The measurement never guesses by trimming trailing `$00` or `$FF` bytes.

The current GearColeco result proves deterministic startup and 120 completed frames, not yet equivalent controller semantics. Controlled input injection and assertions on all five result bytes are the next required QA stage.

For every fixture record source lines, compiler version, build time, ROM bytes, permanent RAM, worst-frame cycles, and runtime result. A smaller ROM that fails visually or changes behavior is not an optimization win.

Ranked Amy gap plan

Rank	Work	Value	Effort	Risk	Decision gate
1	Five-tool sprite/metasprite benchmark	High	Medium	Low	Evidence first; no syntax yet
2	First-class metasprite data/rendering prototype	High	Medium	Medium	Must beat or clarify manual sprite code without hiding priority or scanline limits
3	Integrate Coleco BIOS/Tiny Sound authoring into Studio	High	Medium-large	Medium	Round-trip existing sound tables and preserve byte-exact expert editing
4	ZX1 direct-to-VRAM speed/size experiment	Medium	Medium	Medium	Add only if payload + decoder or cycles wins a documented use case
5	Small explicit animation service	High	Large	Medium-high	Zero linked cost when unused; RAM and per-frame cycle budget must be visible
6	Aggregate-field 2D arrays and remaining operand symmetry	Medium	Medium	Medium	Driven by a real game repro, with fail-closed diagnostics and five-profile tests

Next concrete work: metasprite evidence before language design

1. Define one exact actor made from two or three 8x8 sprites, plus enough independent sprites to exceed four sprites on one scanline.
2. Implement it idiomatically in all seven solutions using only ColecoVision-confirmed APIs.
3. Capture occupied ROM, permanent RAM, SAT update bytes, worst-frame cycles, visual result, and priority/flicker behavior.
4. In Amy, compare explicit `set sprite` code against a data-driven helper prototype.
5. Reject the feature if it adds hidden NMI work, allocates RAM when unused, obscures sprite 0 priority, or makes protected composite actors flicker internally.
6. If the prototype wins, then define syntax, diagnostics, source-debug mapping, autocomplete, highlighting, documentation, and clean-repository synchronization.

Deliberate non-goals

- ROM banking and extra hardware remain outside Amy's stock-console philosophy.
- General dynamic strings remain a poor trade for 1 KB RAM; fixed buffers and formatting are the preferred model.
- Multi-platform abstraction, a general C standard library, and IEEE floating point are not useful measures of Amy's ColecoVision-specific quality.